

설비 고장을 예측하는 AI 데이터 분석

모델 평가와 해석

고장 예측 결과 해석

C O N T E N T S

목차

01 F1-score와 분류 리포트

02 임계값 조정

03 이상탐지 평가와 결과 해석

01

F1-score와 분류 리포트

F1-score란 무엇인가 · 분류 리포트 읽는 순서

F1-score가 필요한 이유

정밀도·재현율 두 숫자를 하나로 묶어 모델을 비교하기 위함

01 · 비교의 어려움

숫자가 두 개씩이면
한마디로 못 정함

모델을 여러 개 비교할 때 정밀도와 재현율을
따로 보면 누가 나은지 모호함

02 · 엇갈리는 모델

A는 정밀도↑ 재현율↓
B는 반대

A는 정밀도 0.9·재현율 0.6, B는 정밀도 0.7·재현
율 0.8 — 어느 쪽이 나은가

03 · F1의 등장

두 지표를 함께 본
하나의 점수

빠른 비교는 F1로, 중요한 결정은 두 지표를 다
시 보는 방식의 균형

조화평균이라는 아이디어

산술평균의 약점 — 한쪽이 0이어도 점수가 높게 나옴

정밀도와 재현율을 합칠 때 단순히 더해서 둘로 나누지 않는 이유
산술평균은 한쪽이 무너져도 점수가 절반쯤 나와 잘못된 평가를 만듦

예시 · 쓸모없는 모델

정밀도 1.0, 재현율 0.0인 모델

고장이라 한 건 다 맞혔지만 실제 고장은 다 놓친 모델

산술평균 = $(1.0 + 0.0) / 2 = 0.5$ — 절반은 한 것처럼 보임

한쪽이 0인데 평균이 0.5라는 분명한 잘못된 평가

그래서 두 지표를 합칠 땐 산술평균이 아닌 조화평균이 필요

조화평균의 성질

둘 중 **작은 값**에 더 끌려가는 평균 — 한쪽이 무너지면 점수도 무너지짐

01 · 한쪽이 0이면

전체도 0

둘 다 좋아야 높은 점수, 한쪽이 0에 가까우면
평균도 0으로 끌려 내려감

02 · 비슷할 때

**산술평균과
거의 동일**

0.9와 0.9 → 조화평균도 약 0.9, 둘이 비슷하면
거의 같은 값

03 · 한쪽이 무너지면

점수도 급락

0.9와 0.1 → 산술평균은 0.5, 조화평균은 약
0.18로 뚝 떨어짐

F1-score란 무엇인가

F1 = 정밀도와 재현율의 **조화평균**으로 만든 하나의 균형 점수

공식

$$F1 = 2 \times (\text{정밀도} \times \text{재현율}) / (\text{정밀도} + \text{재현율})$$

외우기보다 의미를 기억 — 두 지표를 조화평균으로 묶은 점수

01 · 범위

0 ~ 1

1에 가까울수록 두 지표가 둘 다 높다는 뜻

02 · 같은 F1이라도

안쪽 분포는 다름

F1 0.8이어도 정밀도·재현율 분포는 모델마다 다름

03 · 우리 문제에선

재현율도 함께

F1과 함께 재현율을 따로 확인하는 습관

f1_score() 사용법

함수 한 줄로 F1 계산 — 손계산 공식과 결과가 같은지 확인 가능

CODE · F1_SCORE.PY

```
from sklearn.metrics import f1_score, precision_score, recall_score
f1 = f1_score(y_test, y_pred)
P = precision_score(y_test, y_pred)
R = recall_score(y_test, y_pred)
print(f1, 2 * P * R / (P + R)) # 두 값이 같음
```

정답 먼저·예측 나중, 양성=1 기준은 다른 지표와 동일
조화평균 공식 결과와 **f1_score** 결과가 같으면 F1의 작동을 이해한 것

함께 따라 칠 코드는 02_고장_예측_결과_해석_개념코드.py의 섹션 43

F1을 신뢰할 때와 주의할 때

F1은 두 지표에 **똑같은 비중**을 둬 — 한쪽이 더 중요한 문제엔 주의

Q1 · 언제 유용한가

정밀도·재현율 중요도가
비슷하거나 **균형 자체**가 목표

어느 쪽이 더 중요한지 안 정했을 때
여러 모델을 빠르게 줄 세우는 비교 요약이 필요할 때

Q2 · 언제 조심해야 하나

한쪽이 더 중요할 때
(우리 문제는 **재현율 우선**)

재현율이 낮는데 정밀도가 높아
F1이 그럭저럭 나오는 모델을 통과시킬 위험

F1과 함께 재현율을 본다

F1은 **균형 점수**로 쓰되, 최종 판단은 우리 우선순위와 함께

01 · F1 사용 원칙

균형을 한눈에 보는
비교 요약으로 사용

재현율 중시 현장은 F1과 재현율을 함께 확인
F1만 보고 모델을 통과시키지 않기

02 · F-beta 참고

한쪽에 더 무게를 주는
F-beta 변형도 존재

재현율을 더 중시하면 그쪽에 가중치
지금은 'F1은 둘을 똑같이 본다'는 사실만 기억

classification_report 한눈에 보기

정밀도·재현율·F1을 **클래스별**로 한 번에 보여 주는 표

CODE · CLASSIFICATION_REPORT.PY

```
from sklearn.metrics import classification_report
print(classification_report(
    y_test, y_pred,
    target_names=["정상", "고장임박"]
))
```

정확도·정밀도·재현율·F1을 함수 한 번으로 다 출력

클래스별로 보여 주므로 우리는 **고장 임박(1)** 줄을 집중해서 확인
실무에서 모델을 평가할 때 가장 먼저 찍어 보는 함수

클래스별 지표와 평균

support·macro·weighted — 처음 보면 헷갈리는 항목 정리

01 · SUPPORT

각 클래스의
실제 개수

고장 임박의 support가 너무 작으면 그 줄 지표는 우연에 흔들리기 쉬움

02 · MACRO AVG

클래스 개수 무시
단순 평균

소수 클래스도 동등하게 반영, 우리에게 중요한 고장 임박도 공평히 보임

03 · WEIGHTED AVG

개수에 비례한
가중 평균

다수 클래스인 정상에 끌려가 우리에게 다소 위험한 평균

소수 클래스를 공정하게 보려면 macro avg와 고장 임박 줄을 직접 확인

평균에 속지 않는 법

전체 평균 숫자 **하나**에 속지 않기 — 소수 클래스 줄을 직접 보기

위험 · 다수 클래스에 끌리는 평균

불균형 데이터에서 **weighted avg**는 **정상**에 끌려 좋아 보임

accuracy 같은 전체 평균도 다수 클래스에 끌려 좋게 보일 수 있음

해법 · 우리가 볼 곳

macro avg와 **고장 임박(1) 줄**을 직접 확인

소수 클래스인 고장을 공정하게 평가

우리가 볼 곳은 **고장 임박 줄**과 **macro avg**, 이 둘

분류 리포트 읽는 순서

어떤 리포트가 나와도 막히지 않는 5단계 읽기

STEP 1

support 확인

고장 임박 개수가 충분한지 — 너무 적으면 우연에 흔들림



STEP 2

고장 임박 재현율

가장 중요한 지표 — 놓침이 가장 비싼 우리 문제

가장 비싼 실수



STEP 3

같은 줄 정밀도

경보의 신뢰도 — 헛경보 비율을 함께 점검



STEP 4

F1 균형 확인

두 지표의 균형이 어디로 끌려 있는지 한 숫자로 요약



STEP 5

정상·macro로 마무리

정상 줄과 macro avg로 전체 균형을 확인하며 마침표

어디부터 볼지 정해 두면 어떤 리포트를 펼쳐도 당황하지 않는 절차

리포트 읽기 예시

고장 임박(1) 줄을 가장 집중해서 읽기

구분	recall	f1
0 정상	0.97	0.96
1 고장임박	0.75	0.78
macro avg	0.86	0.87

고장 임박 재현율 **0.75** — 실제 고장의 75%를 잡고 25%를 놓침
macro avg는 두 클래스를 공평하게 평균한 값, 전체 균형을 볼 때 참고

읽는 순서가 결론을 바꾼다

가장 작고 중요한 **소수 클래스** **줄부터** 읽기

읽는 순서	❌ 전체 평균부터	✅ 소수 클래스 줄부터
-------	-----------	--------------

눈에 띄는 숫자	weighted f1 0.94	고장 재현율 0.55
----------	------------------	-------------

결론	훌륭한 모델로 통과	위험한 모델임을 포착
----	------------	-------------

현장 영향	고장의 절반 가까이 놓침	미리 알아채 대응 가능
-------	---------------	--------------

어디를 먼저 보느냐가 결론을 통째로 바꿈. 화려한 전체 평균에 현혹되지 말고 소수 클래스 줄부터 냉정하게

실습 — F1 계산과 분류 리포트 해석

F1과 classification_report를 출력하고 5단계 순서대로 해석

목 표

F1과 classification_report 출력 후 순서대로 해석

앞에서 만든 모델의 예측으로 F1을 구하기
리포트를 출력한 뒤 5단계 순서대로 해석 적기

핵심

숫자를 우리 문제 언어로 읽기

"고장을 00% 잡고 경보 정확도는 00%"
한 문장으로 모델 성능 요약

실습 — F1 출력

고장 임박 클래스의 F1을 함수 한 줄로 계산해 출력

CODE · F1_PRACTICE.PY

```
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
print("고장 임박 F1:", round(f1, 4))
```

이 값이 정밀도와 재현율 사이 어디쯤인지 나중에 확인
조화평균의 성질 — F1은 둘 중 **더 낮은 쪽**에 더 끌려 있음

실습 — 리포트 출력과 해석

클래스별 지표를 표로 출력하고 한 줄씩 해석 적기

CODE · REPORT_PRACTICE.PY

```
from sklearn.metrics import classification_report
print(classification_report(
    y_test, y_pred,
    target_names=["정상", "고장임박"]
))
```

해석 순서: support → 고장 임박 재현율 → 정밀도 → F1 → macro avg

확인 질문: F1이 정밀도·재현율 중 더 낮은 쪽에 가깝게 끌려 있는가
조화평균의 성질이 데이터에서 그대로 보임

실습 — 해석 체크포인트

아래 세 가지가 모두 맞으면 리포트 읽기 완성

01 · 위치 확인

고장 임박 줄의
재현율·정밀도 **정확히**

리포트에서 1 고장임박 줄을 찾아 두 숫자를 짚어내기

02 · 약점 판정

정밀도·재현율 중
어느 쪽이 약한가

두 값을 비교해 모델이 어느 방향으로 치우쳐 있는지 판정

03 · 한 문장 요약

"고장을 00% 잡고
경보 정확도 00%"

한 문장으로 요약되면 리포트를 제대로 읽은 것

리포트를 읽었으니, 모델 두 개를 비교할 땐 어떤 지표를 기준으로 골라야 할까

두 모델 비교는 어떤 지표로

비교 기준은 정확도가 아니라 **가장 비싼 실수**를 줄이는 지표

비교 기준	✖ 흔한 선택	✔ 올바른 기준
-------	---------	----------

기준 지표	정확도가 높은 모델	가장 비싼 실수를 줄인 모델
-------	------------	-----------------

불균형에서	다수에 끌려 속음	현장 비용을 반영
-------	-----------	-----------

우리 일차 기준	정확도 / weighted f1	고장 임박 재현율
----------	-------------------	-----------

우리에겐 농침이 가장 비싸므로, 재현율이 일차 기준

모델 비교 순서

정해 둔 잣대로 **모든 모델을 같은 기준에** 줄 세우기

STEP 1

**재현율로
고장 포착력 비교**

가장 비싼 실수인 놓침을 줄이는 능력을
먼저 본다



STEP 2

**비슷하면 정밀도로
경보 신뢰도 비교**

재현율이 비슷한 모델끼리는 헛경보가
적은 쪽을 우선



STEP 3

**종합 판단이 필요하면
F1 참고**

"재현율 가장 높으면서 정밀도도 감당 가능한 모
델" — 분명한 근거

02

임계값 조정

임계값을 조정한다는 것 · 임계값과 정밀도·재현율

예측은 확률에서 나온다

모델은 0/1 이전에 **고장일 확률**을 먼저 계산

STEP 1 · 모델 내부

고장일 확률 계산

예: 어떤 설비의 고장 확률을 **0.82**로 계산



STEP 2 · 임계값 적용

0/1로 판정

0.82 → 충분히 높음 → **1**로 판정

CODE · PREDICT_PROBA.PY

```
# 보통 고장(1) 확률은 두 번째 열에서 꺼냄
proba = model.predict_proba(X_test)[:5, 1]
print(proba[:5])
```

predict_proba()로 확률을 직접 확인 — 두 번째 열이 고장(1) 확률

확률이 주는 힘

확률을 보면 위험의 정도를 구분 — 점검 우선순위 결정

01 · 0/1만 본 경우

0.95짜리와 0.51짜리가
똑같이 '1'

위험 정도를 구분할 단서가 없어 점검 순서를 매길 수 없음

02 · 확률을 본 경우

0.95 설비를 0.55보다
먼저 점검

위험 정도가 보이면 점검 우선순위를 데이터로 결정

주의 · 절대 수치로 맹신 금지

모델 확률은 설비 간 **순서 매기기용** — 0.8이 실제 80% 고장률을 의미하지는 않음

기본 임계값 0.5의 의미

임계값 = 확률을 0/1로 바꾸는 **기준선**

SCIKIT-LEARN PREDICT()의 기본 임계값

threshold = 0.5

고장 확률이 0.5 초과면 1, 이하면 0 — 지금까지 본 모든 지표가 이 기준의 결과

중요한 깨달음

0.5는 절대 정답이 아니라 기본값

이 기준선을 옮길 수 있음 — 옮기면 모델 성격이 바뀜
같은 모델을 **다시 학습하지 않고도** 적극적·신중하게 조절

기준선을 옮긴다는 것의 의미를 다음 슬라이드에서

임계값을 조정한다는 것

기준선을 옮기면 **트레이드오프**가 양쪽으로 갈림

Q1 · 임계값을 0.3으로 낮추면

더 **적극적**으로 경보

확률 0.3만 넘어도 고장이라 판정

재현율 **↑** (농침 감소)

정밀도 **↓** (헛경보 증가)

농침이 비싼 현장에 맞춤

Q2 · 임계값을 0.7로 높이면

더 **신중**하게 경보

아주 확실할 때만 고장이라 판정

정밀도 **↑** (헛경보 감소)

재현율 **↓** (농침 증가)

점검이 비싼 현장에 맞춤

재학습 없이 성격을 바꾼다

모델은 그대로, **판정 기준선만** 이동

임계값 조정은 모델을 다시 학습하는 것이 아닌
이미 계산된 확률을 0/1로 바꾸는 **기준선만 바꾸는 일**

확률이 0.4인 설비	임계값 0.5	임계값 0.3
판정 결과	정상 (0)	고장 (1)
애매한 0.4~0.6 구간	통과	경보 대상

모델 학습엔 시간이 들지만 임계값은 **숫자 하나만** 바꾸면 끝

임계값과 재현율·정밀도의 관계

두 지표가 **시소**처럼 부드럽게 맞바뀜 — 임계값이 그 손잡이

임계값 0.1 — 아주 낮춤

거의 다 고장

재현율 ↑↑

정밀도 **바닥**

모든 설비를 점검 대상으로

그 사이 — 균형 구간

부드럽게 맞바뀜

한쪽이 오르면 다른 쪽은 내려감

모듈 1의 **시소**를 손잡이로 움직임

임계값 0.9 — 아주 높임

확실할 때만

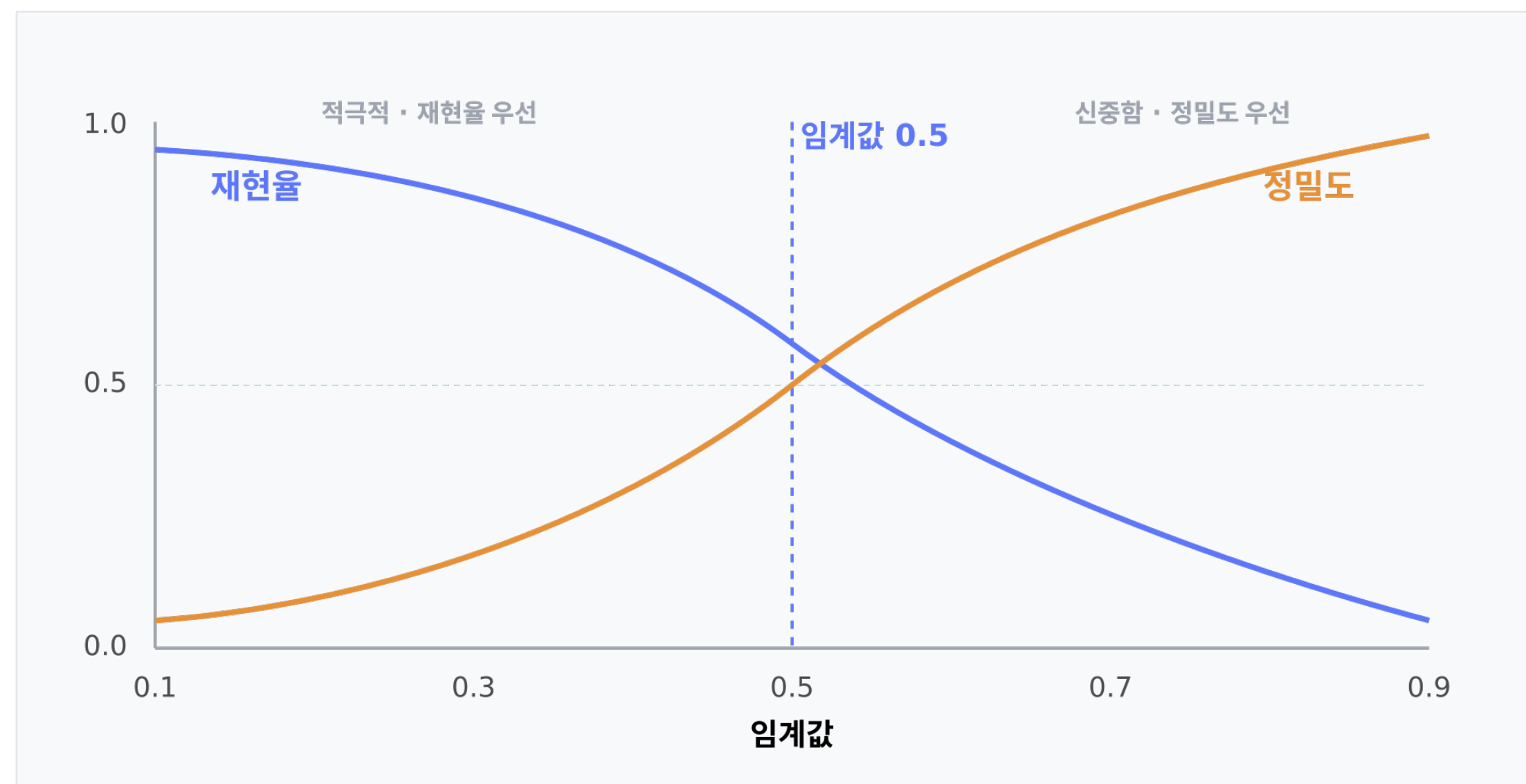
정밀도 ↑↑

재현율 **바닥**

아주 확실한 설비만 점검

임계값-지표 곡선

임계값을 내릴수록 재현율 ↑ · 정밀도 ↓



교차점의 의미

두 선이 엇갈리는 지점이 보통 **F1 최고점**

F1이 가장 높은 균형점이 가까이 있음

하지만

F1 최고점이 항상 정답은 아님

재현율을 더 확보하려면 F1 최고점보다 왼쪽

어느 지점을 택할지는 우리 우선순위가 결정

정답 임계값은 없다

임계값 선택은 기술이 아니라 **의사결정** 문제

01 · 정답의 출처

**비용이
정답을 정함**

재현율·정밀도 중 무엇이 더 가치 있는가에 따라 결정

02 · 판단의 기준

**어느 실수가
더 비싼가**

놓침 비용과 헛경보 비용을 비교해 임계값 지점 선택

03 · 임계값의 역할

**한 모델을
여러 현장에 맞추기**

놓침이 치명적인 라인엔 낮추고, 점검이 비싼 라인엔 높여 조율

그럼 우리 설비 현장에선 어디로 맞춰야 할까

설비에서 임계값을 고르는 기준

농침(FN)이 더 비싼 현장 → 임계값을 0.5보다 낮춤

임계값 0.3 적용 시

확률 0.3만 넘어도 미리 점검 대상

재현율을 높이고 농침을 줄이기 위한 적극적 운영

헛점검 (FP)

늘어남 — 감수

점검 인력·시간이 더 필요해짐

설비 정지 (FN)

줄어듦 — 막음

정지가 훨씬 비싸다면 남는 장사

너무 낮추면 안 되는 이유

적정 임계값 = 재현율 목표와 **점검 가능 범위**의 타협

위험 · 거짓 경보 폭발

지나치게 낮추면 → 경보 신뢰 **붕괴**

거짓 경보가 폭발해 정비팀이 감당 못 함
경보 전체의 신뢰가 무너지는 부작용

적정점 · 재현율 목표 × 점검 가능 범위

재현율을 원하는 수준까지 + 정밀도가 **감당 가능한 선**

하루에 점검 가능한 설비 수가 **현실적 제약**
그 안에서 재현율을 최대화하는 지점이 적정 임계값

실습 — 임계값 조정 실험

확률을 꺼내 임계값을 바꿔 가며 지표 변화 체험

목 표

확률을 꺼내 임계값 0.3·0.5·0.7로
예측 생성

predict_proba로 고장 확률 추출
세 임계값으로 예측을 새로 만들어 비교

핵 심

임계값별 재현율·정밀도 변화 체험

임계값이 내려갈수록 재현율↑·정밀도↓
같은 모델인데 성격이 달라지는 경험

실습 — 확률 추출과 임계값 적용

고장(1) 확률을 꺼내 임계값별 예측 생성

CODE · THRESHOLD_PRED.PY

```
proba = model.predict_proba(X_test)[: , 1]
for t in [0.3, 0.5, 0.7]:
    pred_t = (proba >= t).astype(int)
    print(t, pred_t.sum(), "건 경고")
```

predict_proba에서 **두 번째 열**이 고장(1) 확률
확률 $\geq$ 임계값이면 1, 아니면 0으로 새 예측 생성

임계값이 낮을수록 경고 건수가 늘어남

실습 — 임계값별 지표 비교

각 임계값의 재현율·정밀도·F1 출력

CODE · THRESHOLD_COMPARE.PY

```
from sklearn.metrics import precision_score, recall_score, f1_score
for t in [0.3, 0.5, 0.7]:
    pt = (proba >= t).astype(int)
    print(t, round(recall_score(y_test, pt), 3),
          round(precision_score(y_test, pt), 3))
```

확인 질문 · 임계값을 내릴수록 재현율↑·정밀도↓ 트레이드오프가 보이는가
0.3에서 재현율 가장 높고 정밀도 가장 낮은지 점검

실습 — 임계값 실험 체크포인트

아래 세 가지가 모두 확인되면 임계값을 체득한 것

01 · 임계값 0.3

재현율 **최고**
정밀도 **최저**

적극적으로 경보 → 놓침 감소·헛경보 증가가
데이터에 나타남

02 · 임계값 0.7

정밀도 **최고**
재현율 **최저**

신중하게 경보 → 헛경보 감소·놓침 증가

03 · 깨달음

모델은 그대로
판정 기준선만 이동

같은 모델이 적극적도 신중하게도 변신

실습 — 임계값 후보별 비교표 작성

재현율 목표를 만족하는 **최선의 임계값** 데이터로 선택

목 표

**임계값 0.1~0.9의 지표·FP·FN을
표로 정리**

임계값을 촘촘히 훑으며 재현율·정밀도·F1·FP·FN을 한 표로 작성

핵 심

재현율 목표를 만족하는 최선의 임계값 선택

이 표가 의사결정의 근거 — 막연한 직관이 아닌 데이터 기반 결정

실습 — 비교표 생성

임계값별 지표와 FP·FN을 한 표로

CODE · THRESHOLD_TABLE.PY

```
import pandas as pd
from sklearn.metrics import confusion_matrix, recall_score, precision_score
rows = []
for t in [0.2, 0.3, 0.4, 0.5, 0.7]:
    pt = (proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pt).ravel()
    rows.append([t, recall_score(y_test, pt), fp, fn])
print(pd.DataFrame(rows, columns=["임계값", "재현율", "FP", "FN"]))
```

임계값마다 재현율·정밀도·F1과 거짓 경보(FP)·놓침(FN)을 한 행으로 — 의사결정의 근거

실습 — 임계값 선택과 근거

데이터에 근거해 임계값을 고르고 한 문장으로 정리

01 · 추세 확인

임계값↓ →
재현율·FP↑ FN↓

표 전체에 트레이드오프 패턴이 그대로 보임

02 · 선택 기준

재현율 목표 만족 중
FP 최소

재현율 0.9 이상 임계값 중 거짓 경보가 가장 적은 값

03 · 결론 한 문장

"임계값 00 선택 —
재현율 00·허경보 00건"

데이터에 근거한 임계값 선택 문장

03

이상탐지 평가와 결과 해석

이상탐지 결과 평가 · 정비 의사결정 해석

이상탐지 결과를 라벨로 평가하기

-1/1 출력을 1/0으로 변환 → 혼동행렬.리포트 그대로 사용

STEP 1 · IsolationForest

정상 = 1, 이상 = -1

이상탐지 모델의 출력 형식



STEP 2 · 변환

-1 → 1, 1 → 0

이상=고장=양성=1 형식으로 정렬



STEP 3 · 평가

혼동행렬 그대로

classification_report 그대로 사용

핵심

정답 라벨만 있으면 오늘 배운 방법으로 **똑같이** 평가

출력 형식만 0/1로 맞춰 주면 지도학습 평가 도구를 그대로 재사용

MIMII 라벨로 채점하기

MIMII의 label 컬럼이 **정답 역할** (0 정상, 1 이상)

01 · 재현율

**실제 이상 중
몇 %를 잡았나**

변환한 예측을 label과 비교 — 실제 이상 데이터 중 모델이 잡아낸 비율

02 · 정밀도

**이상이라 한 것 중
몇 %가 진짜였나**

이상으로 분류된 것 중 실제 이상이었던 비율 — 평가 도구는 지도학습과 동일

이상 점수와 임계값

각 데이터에 **얼마나 이상한가** 점수를 매김

01 · `score_samples()`

이상 점수 확인
(**낮을수록 더 이상**)

지도학습의 고장 확률에 대응되는 개념

02 · `contamination`

전체 중 이상 비율 설정
(**민감도 손잡이**)

이상으로 볼 양을 조절하는 손잡이 — 임계값에 해당

contamination은 임계값 역할

지도학습 임계값과 같은 **트레이드오프**가 작동

contamination	크게 (예: 0.2)	작게 (예: 0.05)
성격	더 적극적	더 신중
재현율	↑	↓
정밀도	↓	↑

실제 이상 비율을 모를 때가 많아 **contamination** 정하기가 어려움
MIMII는 라벨이 있어 여러 값을 시험해 볼 수 있는 좋은 환경

도구는 같지만 지도학습과 결정적 차이가 하나 있음

지도학습과 이상탐지 평가의 차이

라벨의 **쓰임**과 모델이 찾은 것의 의미가 다름

Q1 · 라벨을 언제 쓰나

학습엔 안 쓰고
평가 때 **채점용 정답지**로만

지도학습은 라벨을 학습·평가 모두 사용
이상탐지는 정상외의 일반적 모습만 보고 학습

Q2 · '이상'은 곧 '고장'인가

아님 — **센서 오류.**
일시 변동 등도 섞임

이상으로 잡힌 데이터에 진짜 고장 외 다른 원인 혼재
한 단계 더 사람의 해석이 필요

그래서 이상탐지에는 사람의 분류 단계가 한 번 더 필요

이상은 곧 고장이 아니다

이상으로 잡힌 것을 **사람이 한 번 더 분류**

01 · 진짜 고장

즉시 정비 대상

설비에 실제 이상이 발생한 경우 — 우선 점검

02 · 센서 오류

측정 신호 점검

설비가 아닌 센서 자체의 문제 — 다르게 대응

03 · 정비 직후 변화

모델 재학습 필요

정비 직후 신호 변화로 인한 일시적 이상

이 해석 단계가 **이상탐지 결과의 가치를 좌우** — 정밀도가 낮아도 꼭 모델이 나쁜 건 아님

실습 — IsolationForest 결과 평가

MIMII로 IsolationForest 실행 후 라벨과 비교 평가

목표

이상탐지 모델 학습 후
MIMII label과 비교 평가

오늘 배운 평가 도구를 이상탐지에 그대로 적용

핵심

-1/1을 1/0으로 변환 → 혼동행렬.리포트 적용

출력 형식 변환이 모든 평가의 시작점

실습 — IsolationForest 실행

MIMII 특징으로 이상탐지 학습·예측

CODE · ISO_FIT_PREDICT.PY

```
import pandas as pd
from sklearn.ensemble import IsolationForest
dfm = pd.read_csv("mimii_features_sample.csv")
Xm = dfm[["rms", "spectral_centroid", "zero_crossing_rate"]]
raw = IsolationForest(contamination=0.1, random_state=42).fit_predict(Xm)
```

fit_predict 결과는 **정상 1**, **이상 -1**로 나옴

이제 이 출력을 우리 형식인 0/1로 바꿔야 함

fit_predict 한 줄로 학습과 예측 동시에

실습 — 변환과 평가

-1을 1(이상)로 바꿔 라벨과 비교

CODE · ISO_EVAL.PY

```
from sklearn.metrics import confusion_matrix, classification_report
y_true = dfm["label"]
y_iso = (raw == -1).astype(int) # -1(이상)->1, 1(정상)->0
print(confusion_matrix(y_true, y_iso))
print(classification_report(y_true, y_iso, target_names=["정상", "이상"]))
```

raw가 -1이면 1로 바꾸는 **한 줄**로 변환 완료

FN(놓친 이상)·FP(헛경보)를 찾아 한 줄씩 해석

출력 형식만 맞추면 지도학습 평가 도구가 그대로 작동

실습 — 이상탐지 평가 체크포인트

세 가지를 확인하면 이상탐지 평가까지 완성

01 · 형식 확인

변환 후 예측이
0/1 형태인가

혼동행렬·리포트에 그대로 들어갈 수 있는 형식

02 · 놓친 이상

혼동행렬에서
FN이 몇 건인가

실제 이상이지만 모델이 놓친 데이터 건수 확인

03 · 민감도 실험

contamination을
0.1과 0.2로 비교

값을 키울수록 재현율↑·정밀도↓ 확인

비용 관점 해석 — FN과 FP

평가를 **금액의 언어**로 바꾸는 한 줄

예상 손실 공식

$$\text{예상 손실} = (\text{놓침 비용} \times \text{FN}) + (\text{헛경보 비용} \times \text{FP})$$

모델 성능을 경영진이 알아듣는 금액 언어로 환산

FN · 놓침

설비 정지 손실 — 일반적으로 더 비쌈

고장 한 건을 놓쳐 발생하는 비용

FP · 헛경보

추가 점검 비용 — 보통 더 저렴

말썽한 설비를 점검하는 비용

각각의 비용을 정해 곱하고 더하면 예상 손실

비용 환산과 임계값 연결

임계값별 표에 비용을 곱하면 → **예상 손실표**

예시 계산

가정: 놓침 **500만원**, 헛경보 **10만원**

관측: FN 4건, FP 30건

$$500\text{만} \times 4 + 10\text{만} \times 30 = 2,300\text{만원}$$

임계값별 예상 손실표 → 손실 최소 임계값 선택

"이 임계값에서 손실 최소" — **데이터로 정한 결정**

막연한 판단이 아니라 비용 환산을 통한 객관적 의사결정

평가 결과를 정비 의사결정 문장으로

평가의 마지막은 숫자가 아니라 **행동 지침 문장**

"그래서 정비팀은 무엇을 하라"

이 문장이 분석가의 결승점

분석가가 익숙한 언어

F1·재현율·정밀도 숫자

현장 담당자는 F1이 얼마인지 궁금하지 않음

현장이 원하는 언어

무엇을 해야 하는지

숫자를 행동 지침으로 번역하는 단계

좋은 의사결정 문장의 3요소

내일 아침 정비 반장이 **무엇을 할지** 떠오르면 좋은 문장

✕ 나쁜 예

"F1은 0.78, macro 재현율 0.86입니다" — 행동 없음

01 · 무엇을 얼마나

"고장 10대 중
9대를 미리 잡음"

현장 단위(설비 대수·%)로 효과 표현

02 · 무엇을 감수

"멀쩡한 설비 00대도
점검 대상"

트레이드오프의 대가도 솔직히 명시

03 · 행동 + 한계

"우선 점검 +
정상도 정기 점검 유지"

구체적 행동 지침과 모델의 한계를 함께

숫자에서 행동으로 가는 4단계

숫자를 **행동 지침**으로 번역하는 흐름



데이터 분석을 보고서가 아닌 현장을 바꾸는 일로 끌어올리는 습관

좋은 평가 리포트의 구성

이 5요소를 순서대로 답으면 빠짐없는 리포트



핵심 지표 → 혼동행렬 → 비용 해석 → 행동 지침 → 한계 — 다섯 요소면 빠짐없는 평가 리포트

리포트 작성 요령

결론 먼저, 근거는 뒤에

01 · 배치

핵심 지표·행동 지침
앞에 배치

바쁜 담당자가 맨 위 몇 줄만 읽어도 결론 파악

02 · 표현

전문 용어 풀어 쓰고
현장 단위로

숫자는 설비 대수·금액과 함께 제시

03 · 갱신

같은 양식으로
주기적 갱신

시간에 따른 성능 변화도 한눈에 추적

모델의 한계와 재평가 필요성

모델은 **과거 데이터의 세계** 안에서만 정확

01 · 학습 시점

과거 데이터의
패턴을 학습

학습 당시의 운전 조건·설비 구성 기준 — 그 안에서만 정확

02 · 운영 시점

설비·환경이 바뀌면
성능 하락

설비 교체, 운전 조건·계절 변경 등
학습 패턴과 현재 데이터가 어긋남 = **데이터 드리프트**

처음 90%였던 재현율이 몇 달 뒤 뚝 떨어질 수 있음 — 모델은 그대로인데 세상이 변함

주기적 재평가 습관

평가는 한 번이 아니라 **모델이 일하는 내내** 반복

STEP 1 · 측정

월 1회 최근 데이터로
재현율·정밀도 재측정

좋은 습관은 평가를 일정에 넣는 것



STEP 2 · 기준 선

기준(예: 재현율 0.85)
아래로 떨어졌는가

정해 둔 한계선을 데이터로 모니터링



STEP 3 · 조치

재학습 또는
임계값 재조정

모델을 살아 있는 도구처럼 꾸준히 점검하며 운영

실습 — MIMII 데이터에 분류 평가 적용

평가 방법은 데이터가 달라도 동일

목 표

다른 데이터(MIMII)에
지도학습 분류 평가 적용

C-MAPSS가 아닌 MIMII에서도 같은 평가가 작동하는지 확인

핵심

평가 방법은 데이터가 달라도 동일

평가 능력이 데이터 종류에 묶이지 않음을 체득

실습 — MIMII 로드와 분할

MIMII 특징과 라벨로 학습 준비 — 지도학습 모드

CODE · MIMII_SPLIT.PY

```
import pandas as pd
from sklearn.model_selection import train_test_split
dfm = pd.read_csv("mimii_features_sample.csv")
Xm = dfm[["rms", "spectral_centroid", "zero_crossing_rate"]]
ym = dfm["label"]
Xtr, Xte, ytr, yte = train_test_split(Xm, ym, test_size=0.3, random_state=42, stratify=ym)
```

MIMII 특징을 Xm, label을 ym으로 두고 학습용·평가용 분할
여기서는 label을 **학습에도 사용** — 앞 이상탐지 실습과 차이점

stratify=ym으로 클래스 비율을 유지한 채 분할

실습 — 학습과 평가

RandomForest 학습 후 리포트 출력 — **동일 절차**

CODE · MIMII_TRAIN_EVAL.PY

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
clf = RandomForestClassifier(random_state=42).fit(Xtr, ytr)
pred = clf.predict(Xte)
print(classification_report(yte, pred, target_names=["정상", "이상"]))
```

C-MAPSS 때와 **똑같은 함수**, 똑같은 절차

확인 질문 · 정확도가 아니라 이상(1) 줄의 재현율 중심으로 해석

데이터만 바꾸고 평가 코드는 그대로

실습 — MIMII 평가 체크포인트

데이터가 바뀌어도 평가 절차는 동일함을 확인

01 · 절차 일치

**C-MAPSS 때와
같은 함수·절차**

classification_report·confusion_matrix 그대로 사
용

02 · 해석 방향

**이상(1) 비율 확인 +
재현율 중심 해석**

소수 클래스를 먼저 보는 습관이 데이터를 가
리지 않음

03 · 결론

**"평가 방법은 데이터가
달라도 동일하다"**

이 결론을 적어 두면 어디서든 적용 가능

실습 — 종합 평가 파이프라인 구축

반복 평가를 함수로 정리해 재사용

목 표

데이터·모델을 넣으면
지표·혼동행렬이 나오는 함수

모델만 바꿔 넣으면 동일한 평가를 즉시 실행

핵심

반복 평가를 함수로 정리해 재사용

실무에서 이런 함수 하나가 시간을 크게 아낌

실습 — 평가 함수 정의

모델과 평가 데이터를 받는 `evaluate()` 함수

CODE · EVALUATE_FUNC.PY

```
from sklearn.metrics import recall_score, precision_score, confusion_matrix
def evaluate(model, X_test, y_test, name="모델"):
    pred = model.predict(X_test)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
    print(name, "재현율", round(recall_score(y_test, pred), 3), "FN", fn)
```

모델·평가 데이터를 받아 예측·평가를 한 번에 처리
재현율과 FN을 눈에 띄게 보여 주는 게 포인트

가장 비싼 실수(FN)를 강조하는 평가 함수

실습 — 함수로 한 번에 평가

함수에 모델을 넣어 평가 실행 — **모델만 바뀌도** 작동

CODE · EVALUATE_RUN.PY

```
from sklearn.metrics import classification_report
evaluate(model, X_test, y_test, "RandomForest")
print(classification_report(y_test, model.predict(X_test),
    target_names=["정상", "고장임박"]))
```

만든 함수에 우리 모델을 넣으면 평가가 한 번에 작동
모델만 바꿔 넣으면 어떤 모델이든 같은 방식으로 평가
이 함수는 다음 실습인 **모델 비교**에서 그대로 재사용

실습 — 파이프라인 체크포인트

함수가 어떤 모델에도 동작하면 파이프라인 완성

01 · 일관성

모델만 바꿔 넣어도
평가가 동일

이게 함수로 만든 이유 — 평가의 일관성 확보

02 · 가독성

재현율과 FN이
눈에 띄게 표시

가장 비싼 실수를 한눈에 — 우리 우선순위와
정렬

03 · 재사용 정리

다음 실습에서
그대로 재사용

모델 비교 실습에서 그대로 호출 가능하게 정리

실습 — 두 모델 비교 평가

임계값 0.5와 0.3 모델을 **지표·예상 손실**로 비교

목 표

**임계값 0.5와 0.3 두 모델
같은 잣대로 비교**

같은 모델이라도 임계값이 다르면 성격이 다른 두 모델

핵심

현장 가정에 맞는 쪽을 근거와 함께 선택

예상 손실까지 환산해 우리 현장의 답을 결정

실습 — 두 임계값 예측 생성

임계값 0.5와 0.3으로 예측을 만들고 재현율 비교

CODE · TWO_THRESHOLD.PY

```
proba = model.predict_proba(X_test)[: , 1]
pred_05 = (proba >= 0.5).astype(int)
pred_03 = (proba >= 0.3).astype(int)
from sklearn.metrics import recall_score, precision_score
print("0.5", round(recall_score(y_test, pred_05), 3))
print("0.3", round(recall_score(y_test, pred_03), 3))
```

임계값 0.3 쪽 재현율이 더 높음 — 적극적 경보의 효과

실습 — 비용으로 비교

가정 비용으로 예상 손실 비교 — 우열이 **가정에 따라** 달라짐

항목	0.5 기본	0.3 강화
재현율	낮음	높음
놓침 FN	많음	적음
허경보 FP	적음	많음

놓침 비용·허경보 비용을 가정해 각 임계값의 **예상 손실** 계산
가정 비용에 따라 어느 쪽 손실이 작은지 결과가 달라짐 — 값은 실행 결과로 채움

실습 — 모델 선택 체크포인트

"농침 비용이 크므로 임계값 0.3 선택" 근거 문장 완성

01 · 차이 확인

두 임계값의
재현율·FN·FP 차이

숫자로 두 모델의 성격 차이 정량화

02 · 가정의 영향

비용 가정에 따라
우열이 바뀜

예상 손실의 우열은 가정한 비용 구조에 의존

03 · 결정 문장

"농침 비용이 크므로
임계값 0.3 선택"

이유와 결정을 한 문장으로 — 비교 의사결정
의 결승점

실습 — 정비 판단 리포트 작성

선택한 모델 평가를 5요소 리포트 한 장으로 정리

목 표

5요소 리포트 한 장으로 정리

핵심 지표·혼동행렬·비용 해석·행동 지침·한계 — 오늘의 결승점

핵 심

숫자를 현장 언어와 행동 지침으로 번역

그래서 정비팀은 무엇을 할 것인가

실습 — 리포트 5항목

리포트에 담을 5항목

#	항목	담을 내용
01	핵심 지표	고장 재현율·정밀도·F1 (정확도는 참고)
02	혼동행렬	FN·FP를 설비 대수로
03	비용 해석	예상 손실과 임계값 근거
04	행동 지침	고장 표시 설비 우선 점검
05	한계	놓칠 수 있는 비율, 재평가 시점

재현율 중심·현장 단위 숫자·구체적 행동 지침 — 세 가지가 좋은 리포트의 핵심

실습 — 리포트 초안 프롬프트

평가 수치를 정비 리포트 초안으로 정리하는 [AI 프롬프트](#)

AI PROMPT · REPORT_DRAFT

너는 설비 정비 분석가야. 아래 결과를
정비팀용 한 장 리포트로 정리해 줘.
재현율 {재현율} 정밀도 {정밀도} F1 {F1}
놓침 {FN}대 헛경보 {FP}대 임계값 {임계값}
구성: 지표/혼동행렬/손실/행동지침/한계

주의 · AI 초안도 5요소가 다 들어갔는지, **행동 지침**이 분명한지 직접 검토

수치를 글로 옮기기 어렵다면 프롬프트의 값을 채워 AI에게 초안을 요청

실습 — 리포트 작성 체크포인트

"그래서 정비팀은 무엇을 하라"가 **분명히** 있는가

01 · 완비성

5요소를 **모두** 담았는가

핵심 지표·혼동행렬·비용 해석·행동 지침·한계
빠짐없이

02 · 단위

설비 대수·%·금액
현장 단위

모든 숫자가 현장 단위로 적혔는지 확인

03 · 행동 지침

"그래서 정비팀은
무엇을 하라"

이게 없으면 아직 숫자에 머문 것

실습 — 시나리오별 정비 판단 정리

정답은 하나가 아니라 **비용 구조**가 정함

목 표

**현장 조건이 다른 시나리오별로
임계값·우선순위 결정**

세 시나리오로 판단이 어떻게 갈리는지 정리

핵 심

정답은 하나가 아니라 비용 구조가 정함

같은 모델·데이터라도 현장 조건이 다르면 다른 선택

실습 — 세 가지 시나리오

각 경우에 **놓침과 헛경보 중 무엇이 더 비싼가**부터 판단

시나리오 A

**정지 손실 큼,
점검 저렴**

24시간 가동 라인
설비 정지 시 손실이 큰 환경

더 비싼 실수 · 놓침(FN)

시나리오 B

**분해 점검이
매우 비싼 라인**

정밀 가공 라인
한 번 멈춰 분해하는 비용이 큼

더 비싼 실수 · 헛경보(FP)

시나리오 C

**점검 인력 부족 —
하루 점검 수 제한**

제약 조건이 있는 현장
하루 점검 가능 설비 수가 정해짐

제약 안에서 균형

실습 — 시나리오별 판단 방향

정답은 비용 구조가 정함

시나리오	우선 지표	임계값
A · 정지 손실 큼	재현율	낮춤
B · 분해 비쌈	정밀도 신중	너무 낮추지 않음
C · 인력 제한	균형	제약 내 조정

실습 — 최종 정리 체크포인트

평가에 하나의 정답 점수는 없음 — 비용 구조에 따라 달리 쓰는 도구

01 · 비용 판단

시나리오별로
더 비싼 실수 판단

A·B·C 각 시나리오의 비용 구조 정확히 인식

02 · 임계값 방향

그에 맞는
임계값 방향 결정

낮춤·유지·제약 내 조정 — 비용 구조에 맞춰 다르게 선택

03 · 최종 결론

"같은 모델도 현장
비용 구조에 따라
다르게 쓴다"

이 한 문장으로 적으면 이 파트를 완전히 익힌 것

숫자 뒤에 숨은 농침과 헛경보를 읽고 현장의 행동으로 바꾸는 능력 — 평가 여정의 마침표